

06 · Minería de texto de extremo a extremo

Juntamos todo el pipeline sobre un caso real:

Tokenización → Normalización → Representación (TF-IDF) → Análisis

Corpus: las **reseñas de entrega** (`post_compra`), en `datos/resenas_entrega.csv` ,
enriquecidas con el nombre del producto desde `datos/productos.csv` .

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)

import re
import unicodedata

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^[^\w\s]", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
        for t in doc
        if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
len(t.lemma_) > 2
    ]
```

```
def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe([_limpiar(t) for t in textos], batch_size=64)]
```

```
resenas = cargar("resenas_entrega.csv")
productos = cargar("productos.csv")

# Enriquecer con el nombre del producto (join por id) si la columna existe
if "producto_id" in resenas.columns and "id" in productos.columns:
    resenas = resenas.merge(
        productos[["id", "nombre", "categoria"]],
        left_on="producto_id", right_on="id", how="left", suffixes=("", "_prod"),
    )

resenas = resenas.dropna(subset=["texto"]).reset_index(drop=True)
print(resenas.shape)
resenas[[c for c in ["calificacion", "nombre", "texto"] if c in resenas.columns]].head(5)
```

Leyendo ../datos/resenas_entrega.csv (15.0 KB)

Leyendo ../datos/productos.csv (27.5 KB)

(110, 10)

	calificacion	nombre	texto
0	1	OnePlus 12 256GB	Recibi el pedido incompleto, faltaba el cable ...
1	3	Belkin Cargador USB-C 30W	El segundo cargador Belkin llego sin contratie...
2	5	Nintendo Switch Lite	La segunda Switch Lite que regale llego perfec...
3	5	TCL 50 pulgadas 4K Google TV	El segundo television TCL llego perfecto, bien...
4	4	Anker PowerLine III Cable USB-C	Me confirmaron por correo la garantia de un an...

1. Exploración

```
resenas["n_palabras"] = resenas["texto"].str.split().str.len()
print("Reseñas          :", len(resenas))
print("Calificación media :", round(resenas["calificacion"].mean(), 2))
print("Palabras por reseña :", round(resenas["n_palabras"].mean(), 1), "en promedio")
print("\nDistribución de calificaciones:")
print(resenas["calificacion"].value_counts().sort_index())
```

Reseñas : 110
Calificación media : 3.58
Palabras por reseña : 19.3 en promedio

Distribución de calificaciones:

calificacion

1 10
2 15
3 21
4 29
5 35

Name: count, dtype: int64

2. Tokenización + normalización (pipeline reutilizable)

```
resenas["texto_norm"] = normalizar_muchos(resenas["texto"].tolist())  
resenas[["texto", "texto_norm"]].head(3)
```

	texto	texto_norm
0	Recibi el pedido incompleto, faltaba el cable ...	recibi pedido incompleto faltar cable carga or...
1	El segundo cargador Belkin llego sin contratie...	cargador belkin llego contratiempo tiempo esti...
2	La segunda Switch Lite que regale llego perfec...	switch lite regalar llego perfecto justo tiemp...

3. Representación TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
vectorizer = TfidfVectorizer(min_df=3)  
X = vectorizer.fit_transform(resenas["texto_norm"])  
vocab = vectorizer.get_feature_names_out()  
print(f"Matriz TF-IDF: {X.shape[0]} reseñas x {X.shape[1]} términos")
```

Matriz TF-IDF: 110 reseñas x 112 términos

4. Análisis A — ¿qué palabras caracterizan a una reseña negativa vs una positiva?

Separamos por calificación (≤ 2 negativas, ≥ 4 positivas) y, para quedarnos con lo **distintivo** de cada grupo (y no con `llego`, que está en casi todas), ordenamos por la **diferencia** de TF-IDF medio entre grupos.

```
import numpy as np  
  
neg = (resenas["calificacion"] <= 2).values  
pos = (resenas["calificacion"] >= 4).values  
  
media_neg = np.asarray(X[neg].mean(axis=0)).ravel()  
media_pos = np.asarray(X[pos].mean(axis=0)).ravel()  
dif = pd.Series(media_neg - media_pos, index=vocab)  
  
print(f"NEGATIVAS ({neg.sum()} reseñas) - términos que más las distinguen:")  
print(" " + ", ".join(dif.sort_values(ascending=False).head(12).index))
```

```
print(f"\nPOSITIVAS ({pos.sum()} reseñas):")
print(" " + ", ".join(dif.sort_values().head(12).index))
```

NEGATIVAS (25 reseñas) – términos que más las distinguen:
cambio, equipo, tener, pantalla, retraso, pedido, samsung, iniciar, laptop, reclamo, comprar, entrega

POSITIVAS (64 reseñas):
galaxy, xiaomi, perfecto, ningun, configurar, rapido, completo, acer, mes, amazfit, empacado, compra

5. Análisis B — reseñas similares (similitud del coseno)

Sobre los vectores TF-IDF, la similitud del coseno mide cuán parecidas son dos reseñas. Tomamos una y buscamos las 3 más cercanas.

```
from sklearn.metrics.pairwise import cosine_similarity

base = 0
sims = cosine_similarity(X[base], X).ravel()
orden = sims.argsort()[::-1][1:4] # saltamos la propia reseña

print("RESEÑA BASE:", resenas.loc[base, "texto"], "\n")
for j in orden:
    print(f" sim={sims[j]:.2f} | {resenas.loc[j, 'texto']}")
```

RESEÑA BASE: Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar un envío adicional solo para recibir ese accesorio.

sim=0.48 | El OnePlus 12 que recibí no coincidía con el color que había pedido en el pedido original. Tuve que devolverlo y esperar el cambio por el modelo correcto.

sim=0.44 | El Huawei Watch GT 4 llegó a tiempo pero sin el cable de carga dentro de la caja. Tuve que reclamar por separado para que me lo enviaran.

sim=0.42 | El parlante JBL llegó sin el cable de carga incluido en la caja. Tuve que reclamar por separado para que me lo enviaran.

6. Análisis C — bigramas más frecuentes en las reseñas negativas

```
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(ngram_range=(2, 2), min_df=2)
Xn = cv.fit_transform(resenas.loc[neg, "texto_norm"])
frec = Xn.sum(axis=0).A1
top = sorted(zip(cv.get_feature_names_out(), frec), key=lambda x: -x[1][:10])
pd.DataFrame(top, columns=["bigrama (reseñas negativas)", "frecuencia"])
```

	bigrama (reseñas negativas)	frecuencia
0	equipo tecnico	3
1	llego caja	3
2	llego pantalla	3
3	llego retraso	3
4	alto demanda	2
5	cable carga	2
6	caja tener	2
7	caja visiblemente	2
8	color distinto	2
9	comprar llego	2

Cierre y ejercicio propuesto

Con TF-IDF + scikit-learn ya se puede **clasificar automáticamente** una reseña nueva como positiva o negativa:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

y = (resenas["calificacion"] >= 4).astype(int)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print("accuracy:", clf.score(Xte, yte))
```

Los siguientes pasos del temario —**Word2Vec** y los modelos contextuales tipo **BERT / GPT**— sustituyen estos vectores dispersos por *embeddings* densos que además capturan relaciones semánticas.